Homework 4

TAs: Akshan, Xueying, and Siyuan

10-405/605/805: Machine Learning with Large Datasets

Due Wednesday, November 4th at 11:59 PM Eastern Time

Submit your solutions via Gradescope.

IMPORTANT: Be sure to highlight where your solutions are for each question when submitting to Gradescope otherwise you will be marked 0 and will need to submit regrade request for every solution un-highlighted in order for fix it!

Note that Homework 4 consists of two parts: this written assignment and a programming assignment. Remember to fill out the collaboration section found at the end of this homework as per the course policy.

All students are required to complete the following:

1. Written Section 1 *[28 points]*
2. Programming Section 1 *[43 points]*
3. Programming Section 2 *[45 points]*

All students are required to complete **all** sections of the homework. Your homework score will be calculated as a percentage over the maximum points you can earn, which is 116.

Programming: The programming in section 1 of this homework is **NOT** autograded however you are required to upload your completed deliverables to Gradescope. If you do not, you will not receive credit for that programming section.

1 Written [28 Points]

1.1 Simple Neural Network

Consider a feedforward neural network, defined by the following composition of functions:

$$\mathbf{q} = W^{(1)}\mathbf{x} + \mathbf{b^{(1)}} \tag{1}$$
$$\mathbf{h} = ReLU(\mathbf{q}) = \max(\mathbf{q}, 0) \tag{2}$$
$$\mathbf{p} = W^{(2)}\mathbf{h} + \mathbf{b^{(2)}} \tag{3}$$
$$\mathbf{L}(\mathbf{y}, \mathbf{p}) = (\mathbf{p} - \mathbf{y})^T(\mathbf{p} - \mathbf{y}) \tag{4}$$

Here $\mathbf{x} \in \mathbb{R}^D$ is an input observation/data point, $W^{(1)} \in \mathbb{R}^{H^{(1)} \times D}$ is a set of weights corresponding to the first (input) layer, $\mathbf{b^{(1)}} \in \mathbb{R}^{H^{(1)}}$ is a bias term corresponding to the first layer, $W^{(2)} \in \mathbb{R}^{H^{(2)} \times H^{(1)}}$ is a set of weights corresponding to the second layer, and $\mathbf{b}^{(2)} \in \mathbb{R}^{H^{(2)}}$ is a bias term corresponding to the second layer. Note also that the max in step 2 is applied element-wise.

We will consider two ways to build a gradient-descent learning system. The first way will be to find the symbolic gradient of the loss, L, with respect to the weights $W^{(\ell)}$ and biases $b^{(\ell)}$ for $\ell = 1, 2$, using back-propagation. To write down symbolic derivatives, it may be convenient to use the Kronecker delta:

$$\delta_{ij} = \begin{cases} 0 & i \neq j \\ 1 & i = j \end{cases}$$

(a) *[4 points]* Symbolic differentiation for $(\frac{\partial L}{\partial W^{(2)}})$: First, derive $\frac{\partial L}{\partial p_j}$, and $\frac{\partial p_i}{\partial W^{(2)}_{j,k}}$. Then, derive $\frac{\partial L}{\partial W^{(2)}_{j,k}}$ following techniques from lecture. Then write down an expression for the matrix $\frac{\partial L}{\partial W^{(2)}}$.

(b) *[3 points]* Symbolic differentiation for $(\frac{\partial L}{\partial b^{(2)}})$: First, derive $\frac{\partial p_i}{\partial b^{(2)}_j}$. Then, derive $\frac{\partial L}{\partial b^{(2)}_j}$, using already calculated solutions. Then write down an expression for the vector $\frac{\partial L}{\partial b^{(2)}}$.

(c) *[4 points]* Symbolic differentiation for $(\frac{\partial L}{\partial q})$. First, derive $\frac{\partial p_i}{\partial h_j}$ and $\frac{\partial h_j}{\partial q_i}$. Then, derive $\frac{\partial L}{\partial h_i}, \frac{\partial L}{\partial q_i}$ from previously calculated results. Then write down expressions for the vector $\frac{\partial L}{\partial q}$.

(d) *[5 points]* Symbolic differentiation for $(\frac{\partial L}{\partial W^{(1)}}, \frac{\partial L}{\partial b^{(1)}})$: First, derive $\frac{\partial q_i}{\partial W^{(1)}_{j,k}}, \frac{\partial q_i}{\partial b^{(1)}_j}$. Then, derive $\frac{\partial L}{\partial W^{(1)}_{j,k}}, \frac{\partial L}{\partial b^{(1)}_i}$ using already calculated solutions, following techniques from lecture. Then write down expressions for the matrix $\frac{\partial L}{\partial W^{(1)}}$ and the vector $\frac{\partial L}{\partial b^{(1)}}$.

(e) *[2 points]* For a learning rate of alpha, how would you update the parameters $W^{(1)}_{j,k}$ if you were using stochastic gradient descent on the input $\mathbf{x}^*, \mathbf{y}^*$? The expression should be in terms of $\mathbf{x}^*$ and $\mathbf{y}^*$ but you may also use other intermediate computations like $\mathbf{q}^* = W^{(1)}\mathbf{x}^* + \mathbf{b}$, $\mathbf{h}^*$, and etc.

(f) *[2 points]* Define a Wengert list with 4 steps, using the equations from Equations 1-4, using output letter q, h, p, and L. (Hint: You will need a function with 3 inputs for steps 1 and 3).

(g) *[4 points]* What are the backward functions (partial gradients with respect to an input) that must be computed to derive updates for the parameters? For each function, also indicate if it is equal to any symbolic derivative computed above.

For example, one of these backward functions would be

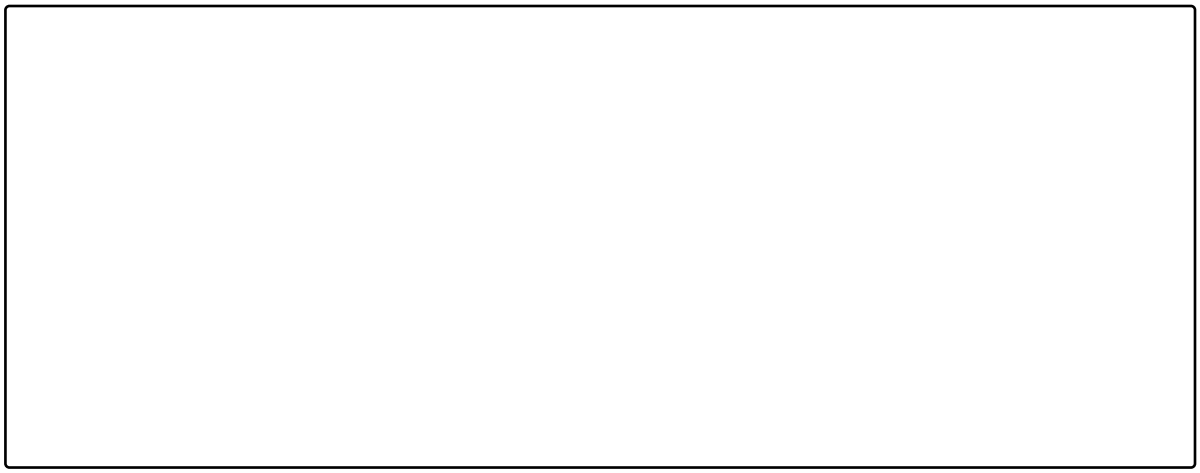
$$\frac{\partial \textbf{squaredError}}{\partial p_j} = \frac{\partial L}{\partial p_j} = 2(p_j - y_j)$$

(h) *[2 points]* Let's define:

$$\mathbf{x} = \begin{bmatrix} 2 \\ 7 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 4 \\ 5 \\ 8 \end{bmatrix}, W^{(1)} = \begin{bmatrix} 3 & 8 \\ 4 & 5 \end{bmatrix}, b^{(1)} = \begin{bmatrix} 4 \\ 10 \end{bmatrix}, W^{(2)} = \begin{bmatrix} 11 & 4 \\ 8 & 13 \\ 6 & 10 \end{bmatrix}, b^{(2)} = \begin{bmatrix} 9 \\ 2 \\ 15 \end{bmatrix}$$

Using these values and results from (f), find the values $\frac{\partial L}{\partial W^{(1)}_{1,1}}$ and $\frac{\partial L}{\partial b^{(1)}_{2}}$.

(i) *[2 points]* How do your experiences from 1.1 help motivate the benefits of DL frameworks like TensorFlow or PyTorch. [note: this can be a 1-2 sentence answer]



2 Programming Section 1: LoRA [43 points]

2.1 Introduction

In this section, you will implement three distinct fine-tuning techniques for a pre-trained large language model (LLM): comprehensive fine-tuning, fine-tuning of the final layer, and **Lo**w-**R**ank **A**daptation (LoRA).

For large pre-trained models, fully retraining all parameters is often impractical due to the substantial demands on training time and memory. LoRA offers a parameter-efficient fine-tuning approach that allows for the adaptation of large pre-trained models with minimal computational overhead. LoRA accomplishes this by integrating trainable low-rank decomposition matrices into the model's architecture, focusing specifically on the layers of the Transformer model.

2.2 Dataset

In this experiment, you will fine-tune your model on the **S**tanford **S**entiment **T**reebank (SST), a widely used dataset for sentiment analysis in natural language processing. Introduced by Socher et al. in 2013, it is based on Rotten Tomatoes movie reviews and consists of 11,855 sentences parsed into 215,154 unique phrases. SST offers both fine-grained (SST-5) and binary (SST-2) sentiment labels. SST-5 includes five categories: very negative, negative, neutral, positive, and very positive, while SST-2 simplifies this to positive and negative by excluding neutral sentences. This dataset serves as a benchmark for evaluating NLP models' ability to assess sentiment by considering both individual words and sentence structure. In this experiment, you will use SST-5.

2.3 Getting lab files

After downloading and unzipping the `Source code (zip)` from the course website, you can obtain a the HW4 starter notebook. The `hw4_part1.ipynb` notebook is the file you will modify. Follow the `TODO` tags within the notebook and complete the instructions to finish the required code. You will only need to upload the completed notebook to Gradescope.

To run the notebook:

1. Upload the folder to your Google Drive.
2. Open `hw4_part1.ipynb` using Google Colaboratory (Colab).

2.4 Grading

Programming section 1 will be graded as follows:

- Empirical and free-response questions in this document [33 points]
- Manual grading of code on Gradescope [10 points]

It is strongly advised to begin your Programming Section 1 early, as running all experiments (excluding coding and debugging) on Google Colab with a T4 GPU is estimated to take approximately 3 hours. Additionally, it is recommended to code and debug on your personal computer before training on Google Colab to avoid exceeding the daily GPU time limit.

2.5 Low-Rank Adaptation (LoRA)

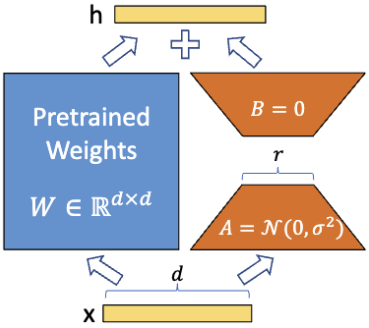


Figure 1: Illustration of Low-Rank Adaptation (LoRA)

In traditional fine-tuning methods, all model parameters are updated, which can be computationally expensive, especially for large models. LoRA addresses this issue by introducing a more efficient approach that reduces the number of parameters updated during fine-tuning. This makes the process more feasible for environments with limited resources.

LoRA restricts the update of the weight matrix $W_0 \in \mathbb{R}^{d \times k}$ using a low-rank decomposition:

$$W_0 + \Delta W = W_0 + BA$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, with $r \ll \min(d, k)$. Only the matrices A and B are trainable, significantly reducing the number of updated parameters. The pre-trained weight matrix W_0 remains unchanged during adaptation, which helps preserve the model's learned knowledge. To maintain stability at the beginning of training, the parameters A and B are initialized in a specific way: B is initialized to zero, while A is initialized using Kaiming initialization (also known as He initialization). This ensures that the initial update $\Delta W = BA = 0$, allowing the model to begin training in a stable state without immediately introducing large changes to the pre-trained weights.

To ensure a balanced update, the low-rank modification is scaled by a factor α/r:

$$h = W_0 x + \frac{\alpha}{r} BAx$$

where x is the input to the layer, α is a constant in r. The scaling factor α allows for proper control over the influence of the low-rank update relative to the pre-trained weights.

LoRA is highly effective for adapting large pre-trained models, such as GPT-2 or GPT-3, to specific tasks while minimizing computational costs. It enables fine-tuning for applications such as sentiment analysis or text classification while preserving the model's general-purpose knowledge. By leveraging LoRA, large models can be efficiently adapted to new tasks and domains with limited computational resources or small datasets. This makes it a valuable tool in modern machine learning pipelines, particularly for deploying models in resource-constrained settings.

Before starting this coding experiment, it is highly recommended to review the following resources to gain a deeper understanding of LoRA: Low-Rank Adaptation of Large Language Models (LoRA) - Paper and Introduction to LoRA - YouTube Video.

2.6 Attention in Transformer-based Models

In this homework, you will be experimenting with the "bidirectional encoder representations from transformers" (BERT) model. BERT is a transformer-based language model (LM) that relies heavily on the attention

mechanism as a core component. The model contains numerous matrix multiplication operations, which are suitable points to apply LoRA.

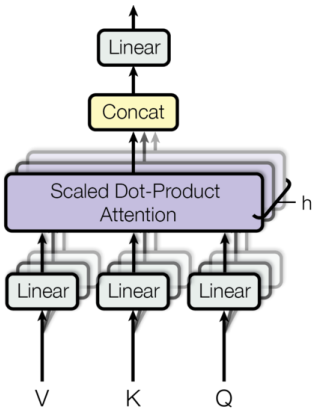


Figure 2: Illustration of the multi-head scaled dot-product attention mechanism.

In the multi-head attention layer of a transformer, a query (Q) and a set of key-value pairs (K and V) are used to compute an output. The output is a weighted sum of the values, where the weights are determined by the compatibility between the query and the keys. This process is mathematically defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

For the purpose of the assignment, it is sufficient to focus on identifying where matrix multiplications occur in the attention mechanism, and apply the LoRA technique to those matrices. However, if you're interested in learning more about transformers and the attention mechanism, you can explore the paper Attention Is All You Need (2017) and the following blogs by an OpenAI researcher: The Transformer Family by Lilian Weng (2020) and Attention Mechanism by Lilian Weng (2018).

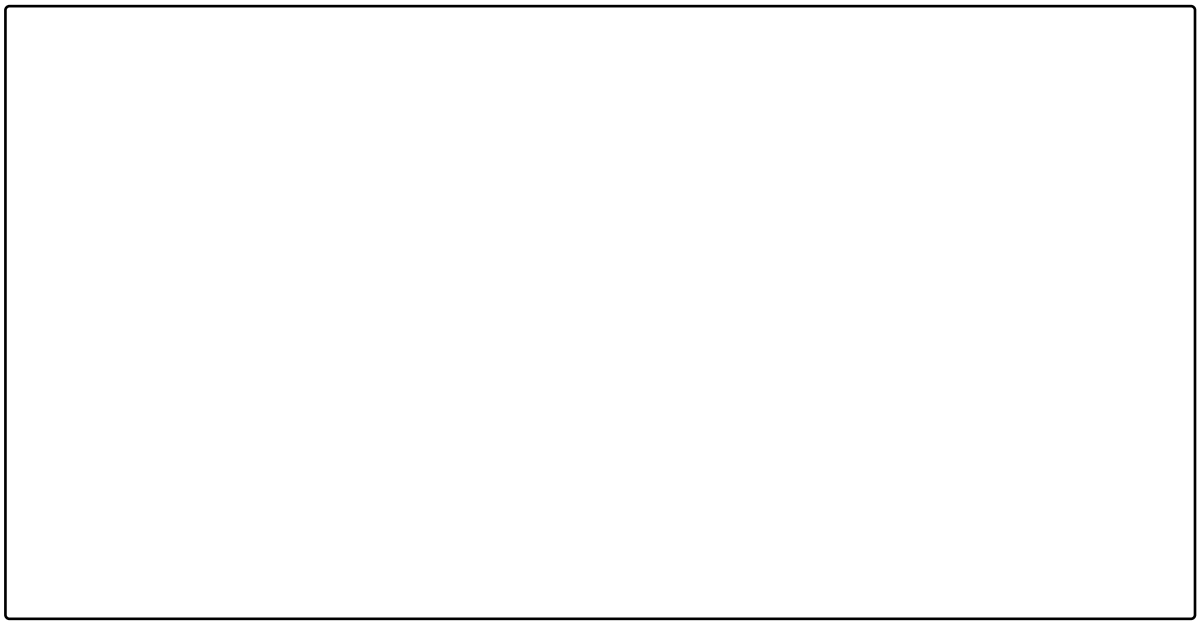
2.7 Questions

Please run full fine-tuning, last layer fine-tuning, and LoRA fine-tuning (add LoRA to all query and value weights) with 20 epochs, learning_rate $= 1 \times 10^{-4}$, $\alpha = 32$, $r = 16$, lora_dropout $= 0.4$, then answer questions (a) - (e).

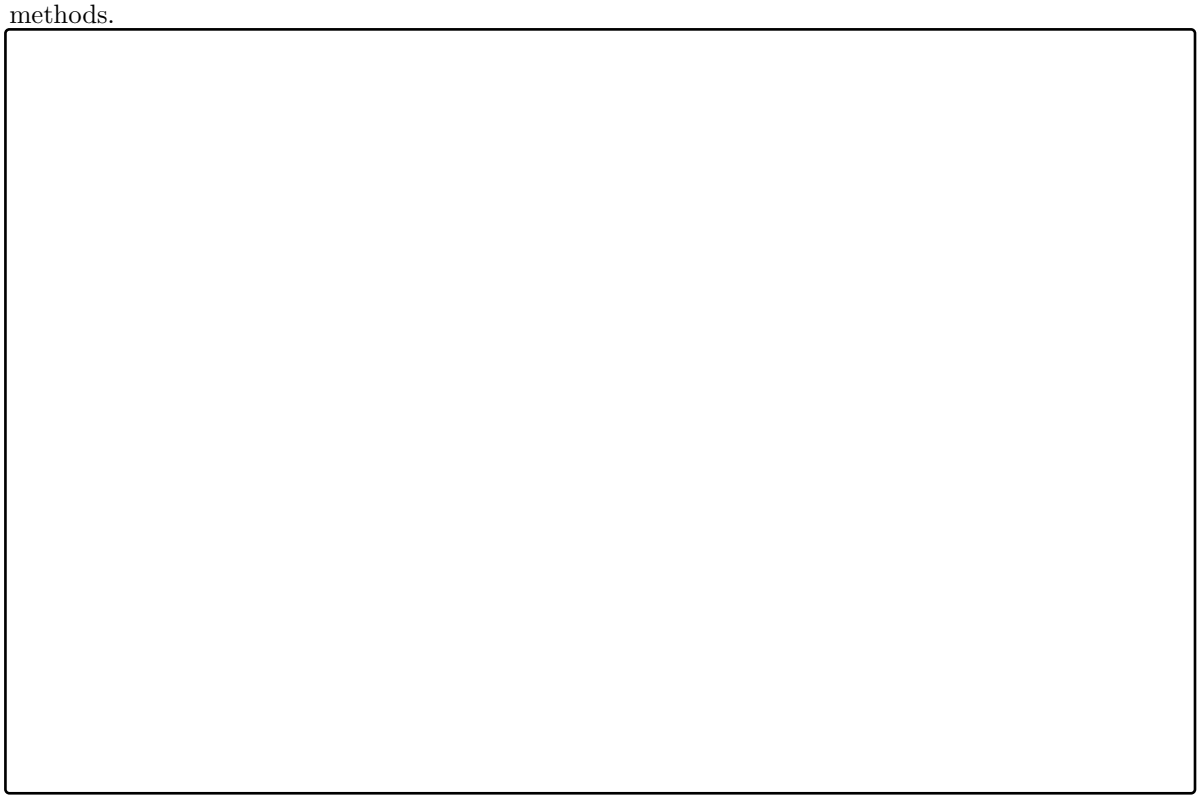
(a) *[6 points]* Plot three pie charts to illustrate the distribution of trainable parameters compared to the total parameters for the three fine-tuning methods.

(b) *[3 points]* Plot a line graph illustrating the training loss against epochs for the three fine-tuning methods.

(c) *[3 points]* Plot a line graph illustrating the val/dev accuracy against epochs for the three fine-tuning methods.



(d) *[3 points]* Plot a bar chart comparing validation accuracy and F1 score for the three fine-tuning methods.



(e) *[4 points]* Based on your earlier visualizations, how does the performance of fine-tuning with a LoRA model compare to full fine-tuning and last layer fine-tuning? Please provide a brief discussion, including comments on the convergence analysis of validation loss.

(f) *[6 points]* Please run full fine-tuning, last layer fine-tuning, and LoRA fine-tuning (add LoRA to all query and value weights) with 20 epochs, learning_rate $= 1 \times 10^{-4}$, $\alpha = 32$, lora_dropout $= 0.4$, and ranks of $\{1, 16, 256\}$, then report the best validation accuracies (NOT the last round validation accuracies) for each rank.

Rank (r)	**Best Test Accuracy**
1	
16	
256	

(g) *[4 points]* How does increasing the rank r, while keeping the scaling factor α constant, affect the model's performance? What might be the reason for this behavior? Based on the results, what trade-offs might be involved in choosing different ranks for fine-tuning a model using LoRA?

(h) *[4 points]* In your implementation, does LoRA lead to an increase in forward propagation time at training stage compared to full fine-tuning? If so, can it be eliminated? If not, please explain the reason. How about during inference time?

Hint: $h = W_0 x + \frac{\alpha}{r} BAx = (W_0 + \frac{\alpha}{r} BA)x$.

3 Programming Section 2: From URLs to Logits, part 1 [45 points]

Introduction

In this section, we will kick off a multi-part series that will involve pretraining a GPT2-class language model from scratch on a dataset curated and preprocessed by you. You will be involved in (nearly) every step of the data preparation pipeline using raw data scraped from the web by selecting a subset of the common crawl dataset that you wish to use. We hope that seeing the full pretaining pipeline, end-to-end, will demystify some of the "magic" of LLMs and allow you to gain a much deeper understanding of how these systems actually work.

Getting Started

3.0.1 Common Crawl

Common Crawl is a massive dataset consisting of petabytes of data scraped from the web. The corpus is commonly used in training LLMs, including in training most of your favorite AI chatbots and LLMs. For example, 80% of the training data used to train GPT-3 came from Common Crawl [1].

[1] https://facctconference.org/static/papers24/facct24-148.pdf

3.0.2 AWS Athena

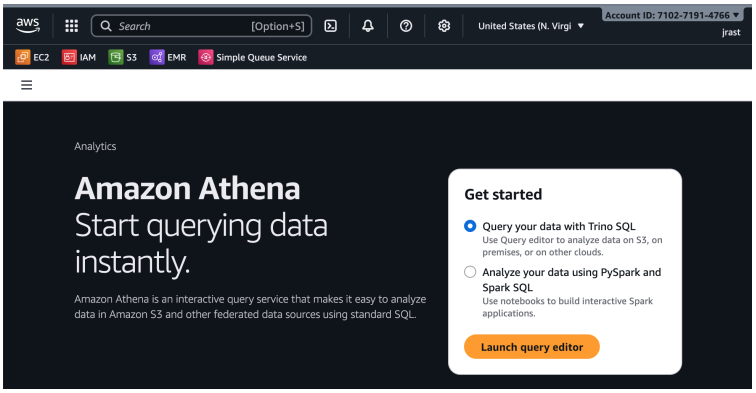
AWS Athena is a AWS service used to run serverless SQL queries on data stored on S3. Because the Common Crawl dataset and its index are accessible for free on AWS, we can use Athena to query for documents relevant to a task of interest.

To get started using AWS Athena for Common Crawl, please see the guide below:

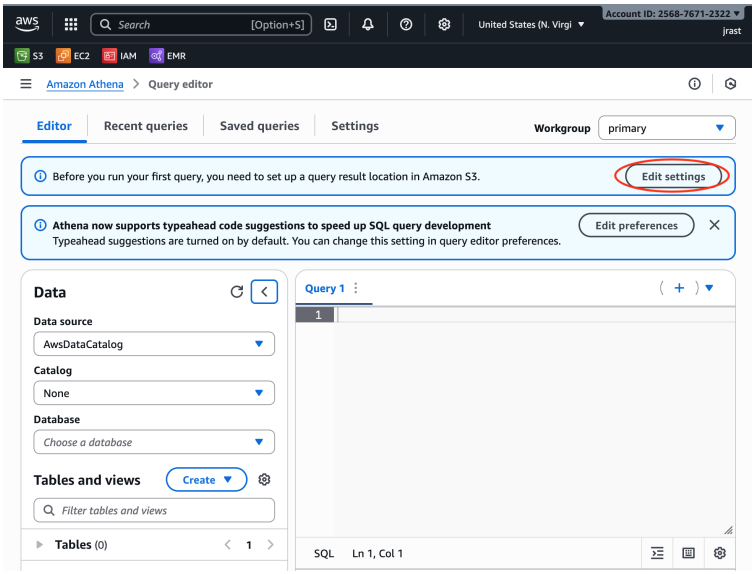
1. **Configuration**

   (a) Create an S3 bucket in the "us-east-1" region to store query results. See the AWS recitation for more details on bucket creation.

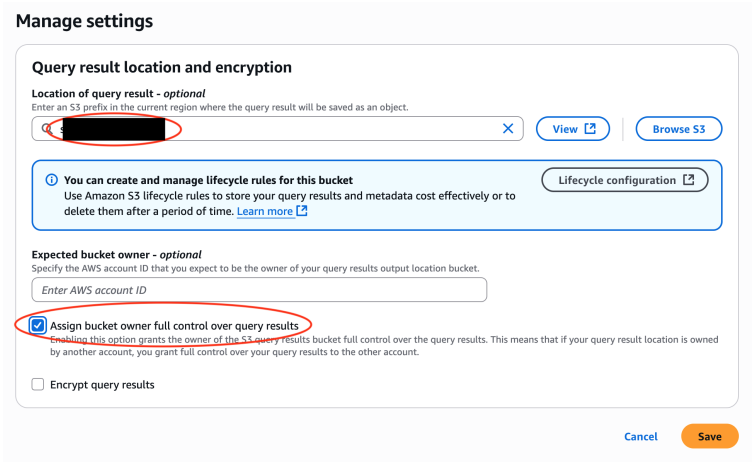
   (b) Navigate to Athena in the AWS Console



   (c) Click on "Launch Query Editor", then "Edit Settings" in the pop-up

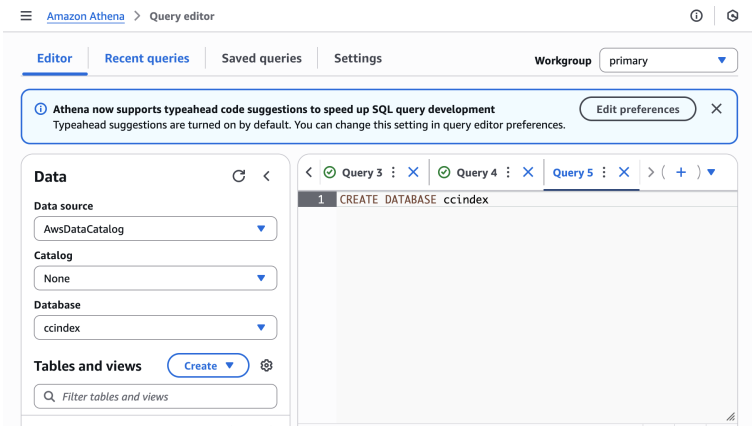


(d) Enter the S3 bucket you created in step 1 (a) and click "Assign bucket owner full control over query results"



2. **Making a Common Crawl table in Athena**

(a) Go to the Athena Query editor



(b) Run the following commands, each with their own query:

i. Create the database:

```
CREATE DATABASE ccindex
```

ii. Create the external table:

```
CREATE EXTERNAL TABLE IF NOT EXISTS ccindex (
 url_surtkey                   STRING,
 url                           STRING,
 url_host_name                 STRING,
 url_host_tld                  STRING,
 url_host_2nd_last_part        STRING,
 url_host_3rd_last_part        STRING,
 url_host_4th_last_part        STRING,
 url_host_5th_last_part        STRING,
 url_host_registry_suffix      STRING,
 url_host_registered_domain    STRING,
 url_host_private_suffix       STRING,
 url_host_private_domain       STRING,
 url_protocol                  STRING,
 url_port                      INT,
 url_path                      STRING,
 url_query                     STRING,
 fetch_time                    TIMESTAMP,
 fetch_status                  SMALLINT,
 content_digest                STRING,
 content_mime_type             STRING,
 content_mime_detected         STRING,
 content_charset               STRING,
 content_languages             STRING,
 warc_filename                 STRING,
 warc_record_offset            INT,
 warc_record_length            INT,
 warc_segment                  STRING)
PARTITIONED BY (
 crawl                         STRING,
 subset                        STRING)
```

```
STORED AS parquet
LOCATION 's3://commoncrawl/cc-index/table/cc-main/warc/';
```

iii. Repair the table:

```
MSCK REPAIR TABLE ccindex
```

3. **Your First Athena Query**

Run the following query. You should see the output "270374".

```
SELECT COUNT(*) AS count
FROM "ccindex"."ccindex"
WHERE crawl = 'CC-MAIN-2025-38'
  AND subset = 'warc'
  AND url_host_registered_domain = 'youtube.com'
```

4. **Querying for Domains**

Finally, we can start looking for documents matching specific domains!

(a) Upload a `domains.txt` file to your S3 bucket. The file should be a newline-separated list of domains, one per line, without any leading "www" or "http".

(b) Run the following query:

```
CREATE EXTERNAL TABLE my_domains (
 domain STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\n'
STORED AS TEXTFILE
LOCATION '[YOUR_S3_BUCKET_HERE]'
```

(c) Now get a list of all English-language documents from these domains!

```
SELECT
   url,
   warc_filename,
   warc_record_offset,
   warc_record_length
FROM "ccindex"."ccindex"
WHERE crawl = 'CC-MAIN-2025-38'
 AND subset = 'warc'
 AND url_host_registered_domain IN (SELECT domain FROM my_domains)
 AND content_languages = 'eng'
```

3.1 Working with Common Crawl [5 points]

3.1.1 Using the Common Crawl index [1 point][LLM-allowed, even in slot A]

Use AWS Athena and the Common Crawl index to query for the location of this document ("Hello, World! program") in the crawl `CC-MAIN-2025-38`. What is the WARC file name, record offset, and record length where this document is stored in the crawl?

3.1.2 Downloading a single WARC record [1 point][LLM-allowed, even in slot A]

Now, download the WARC file containing this record. What is the record ID of the "Hello, World! program" Wikipedia page?

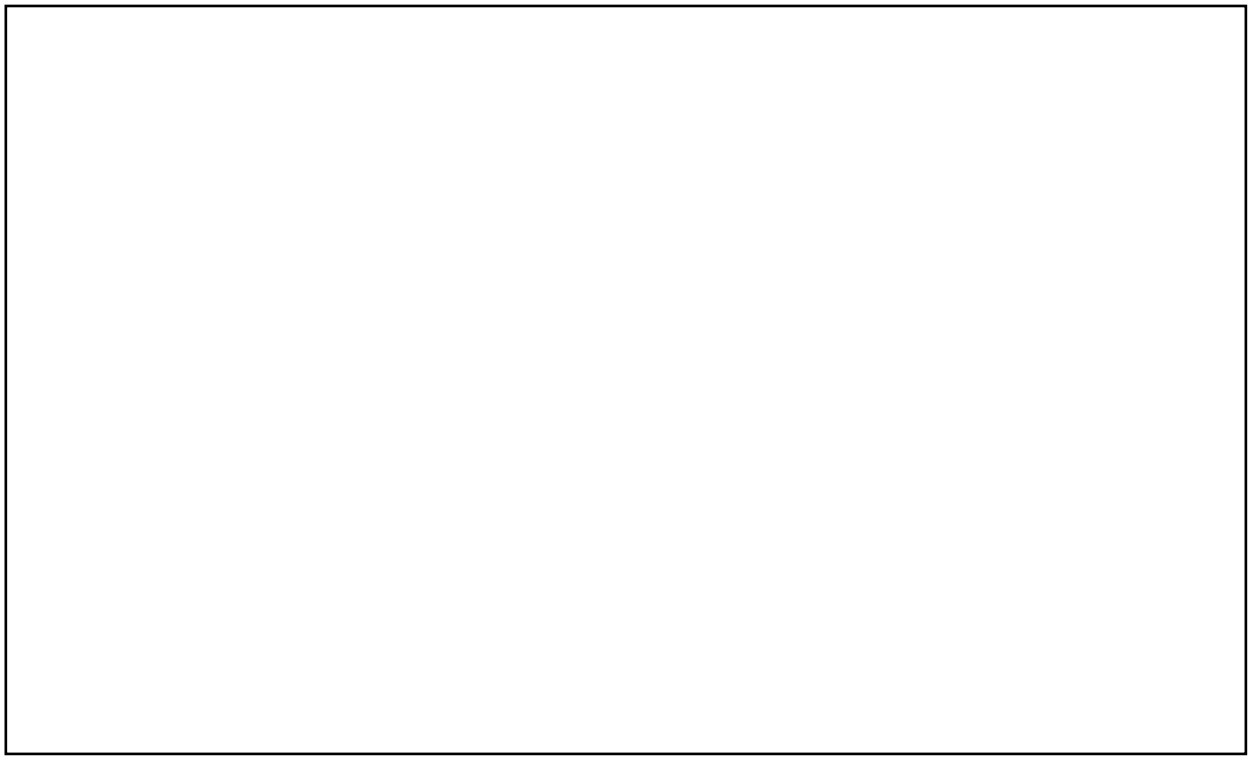
3.1.3 WARC to HTML conversion [1 point][LLM-allowed, even in slot A]

Next, programtically extract the HTML from the WARC file and save it as a standalone .html file. Open it using your favorite web browser and take a screenshot (note: this should NOT be a screenshot of the live web page, but rather a screenshot of the archive found in the WARC file).



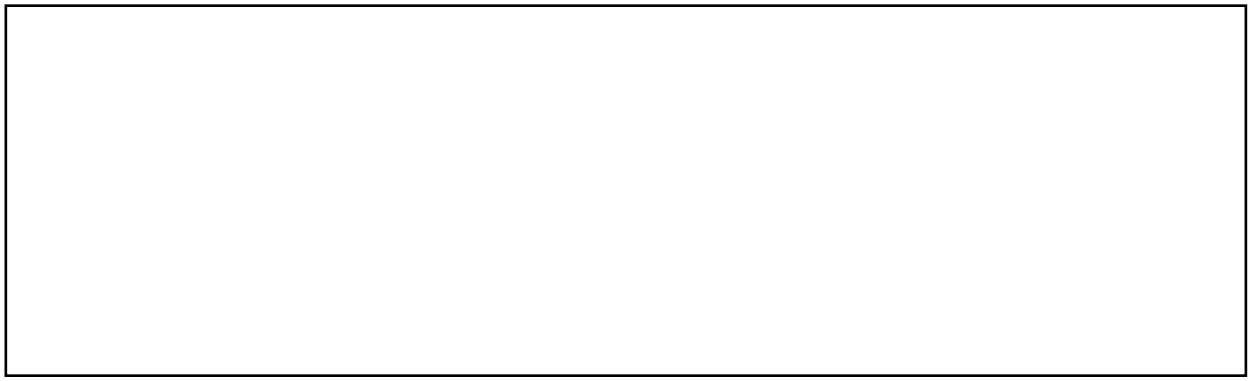
3.1.4 HTML plain text conversion [1 point][LLM-allowed, even in slot A]

Finally, convert the HTML file to a plain text document. You should now have a single Python file that can download, extract, and convert to plain text a document from Common Crawl. Take a screenshot of the plain text and submit it here.



3.1.5 Reflection [1 point]

Do you notice anything noteworthy about the final text used to train an LLM? What could be done to improve the quality of this training corpus?



3.2 MinHash and Locality Sensitive Hashing [40 Points]

Introduction

After collecting a corpus of web-scraped documents, the next step in preparing a LLM pre-training corpus is document quality filtering and deduplication. In this section of the assignment, you will revisit the problem of document similarity with a lens on document de-duplication for LLM training corpus pre-processing. As we saw in Homework 1, a naive approach to calculating document similarity involves comparing every document pair, requiring $O(n^2)$ comparisons. This becomes prohibitively expensive as datasets grow.

This time, you will explore scalable methods for solving the problem. This assignment introduces two key algorithmic techniques that solve this scalability problem: MinHash and Locality-Sensitive Hashing (LSH). These methods transform the similarity search from an exhaustive comparison into an efficient approximate solution.

Compute Requirements and Getting Lab files

You can obtain the notebook `'HW4_part2.ipynb'` in the homework 4 handout .zip file. We recommend working on the notebook locally. You will need to submit queries on Polars Cloud.

Preparing for submission

We provide several public tests via `assert` in the notebook, which will help you know if you are completing the notebook correctly. You will want to pass all those tests before submitting your homework.

Hints and tips

- Keep your total AWS spending below $20. Monitor your usage and configure budget alerts below this limit. Alerts may be delayed and do not automatically stop running resources.
- Pass all local checks before starting a cloud cluster. Test your cloud pipeline on `VERY_SMALL`, then `SMALL` and `LARGE`, before attempting `FULL`. Use a small cluster for debugging; increase `CLUSTER_SIZE` for the final run. When changing the dataset size, rerun the configuration, scan, and pipeline-planning cells.
- The full deduplication run should take less than 30 minutes.
- In the cloud convergence loop, execute each label update and use the completed result's `.lazy()` table as the input to the next iteration. Simply constructing another lazy query does not save intermediate results and can cause earlier iterations to be recomputed. Reuse the materialized edge table throughout the loop.
- Keep the corpus, edges, and component labels on the cluster. Check convergence remotely by counting the IDs whose labels changed. You do not need to download the complete label tables to your computer.
- Bound your convergence loop using `MAX_LABEL_UPDATES`. If labels are still changing after this many updates, raise an error instead of saving an unfinished result. Print the iteration number and number of changed labels to help diagnose unexpected behavior.

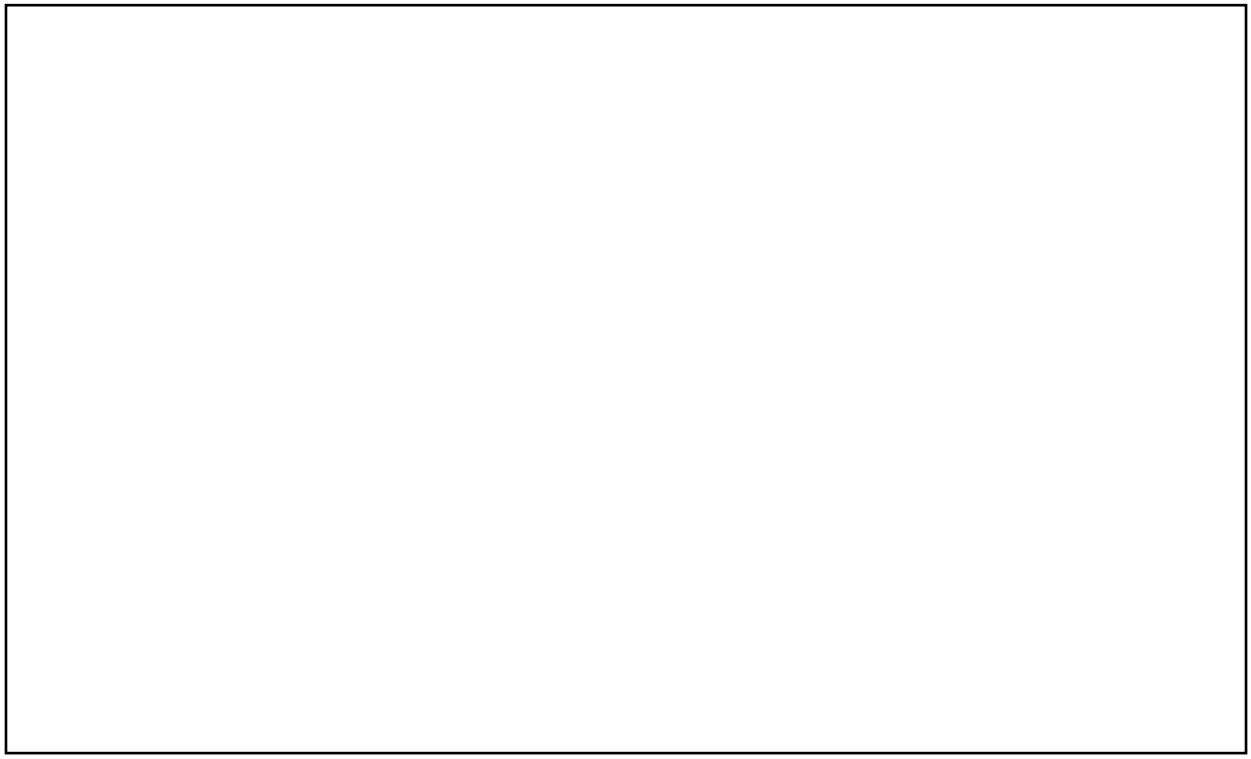
- Preserve the provided error-handling and cluster-shutdown code. Stop the cluster when you finish or pause your work, and verify its status in the Polars Cloud dashboard. After an error stops the cluster, restart from Part 3.1.

3.2.1 Screenshot of deduplicated corpus in S3 *[5 points]*

After successfully deduplicating the `FULL` dataset, open the S3 folder corresponding to your final run. Select this folder and choose **Actions** → **Calculate total size**.

Submit a screenshot showing the output folder's S3 path, total number of objects, and total storage size. Make sure your AWS username (which should be your Andrew ID) is clearly visible; you may include a second screenshot with the account menu expanded if necessary.

Additionally, please include a screenshot of your notebook showing the number of documents retained from the full corpus:



3.2.2 MinHash and LSH Autograded Notebook *[35 points]*

Submit your completed notebook for autograding!

Warning: unlike previous homeworks, please make sure the notebook has been run and that all assert statements are passed before submitting.

4 Collaboration Questions

1. (a) Did you receive any help whatsoever from anyone in solving this assignment?

   (b) If you answered 'yes', give full details (e.g. "Jane Doe explained to me what is asked in Question 3.4")

2. (a) Did you give any help whatsoever to anyone in solving this assignment?

   (b) If you answered 'yes', give full details (e.g. "I pointed Joe Smith to section 2.3 since he didn't know how to proceed with Question 2")

3. (a) Did you find or come across code that implements any part of this assignment?

   (b) If you answered 'yes', give full details (book & page, URL & location within the page, etc.).